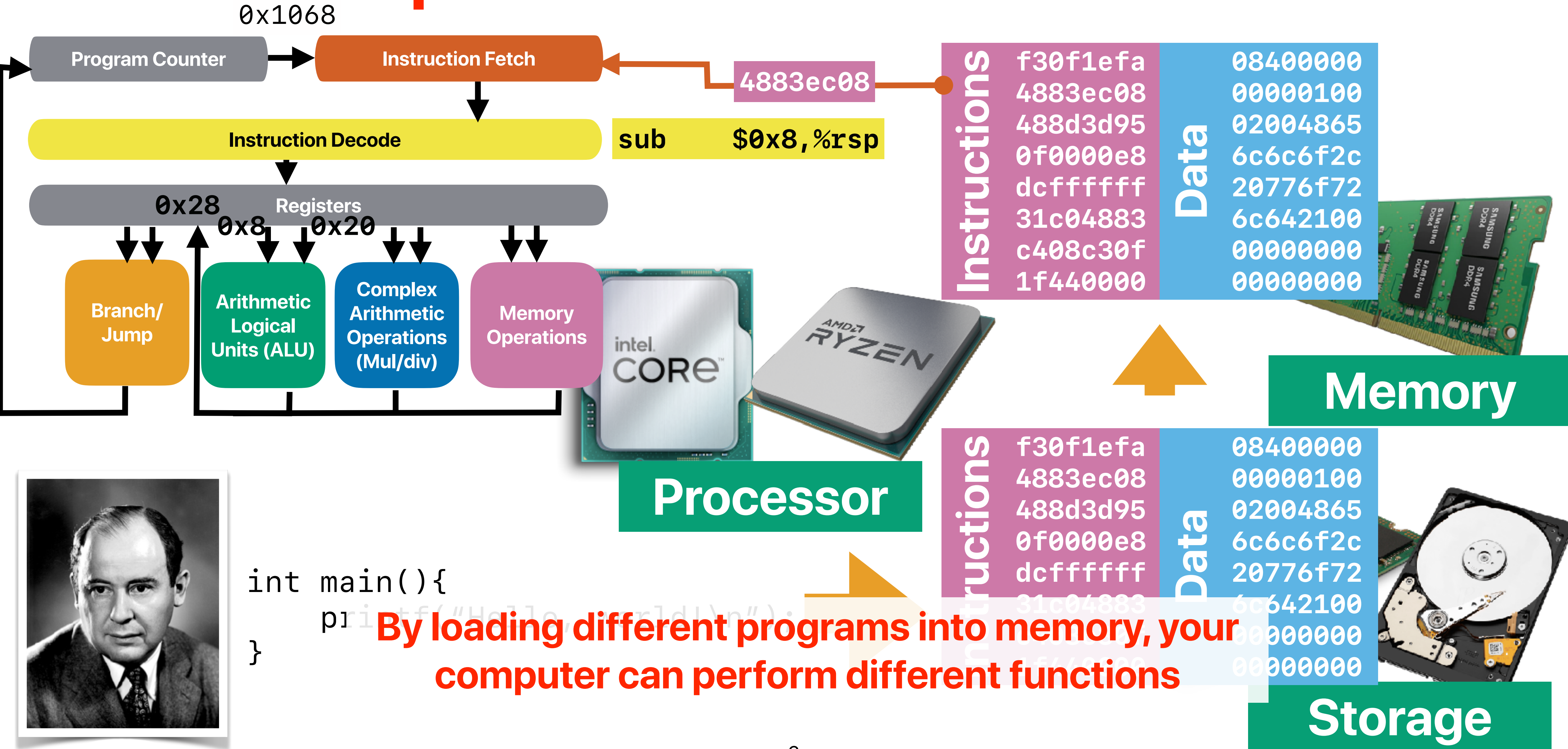


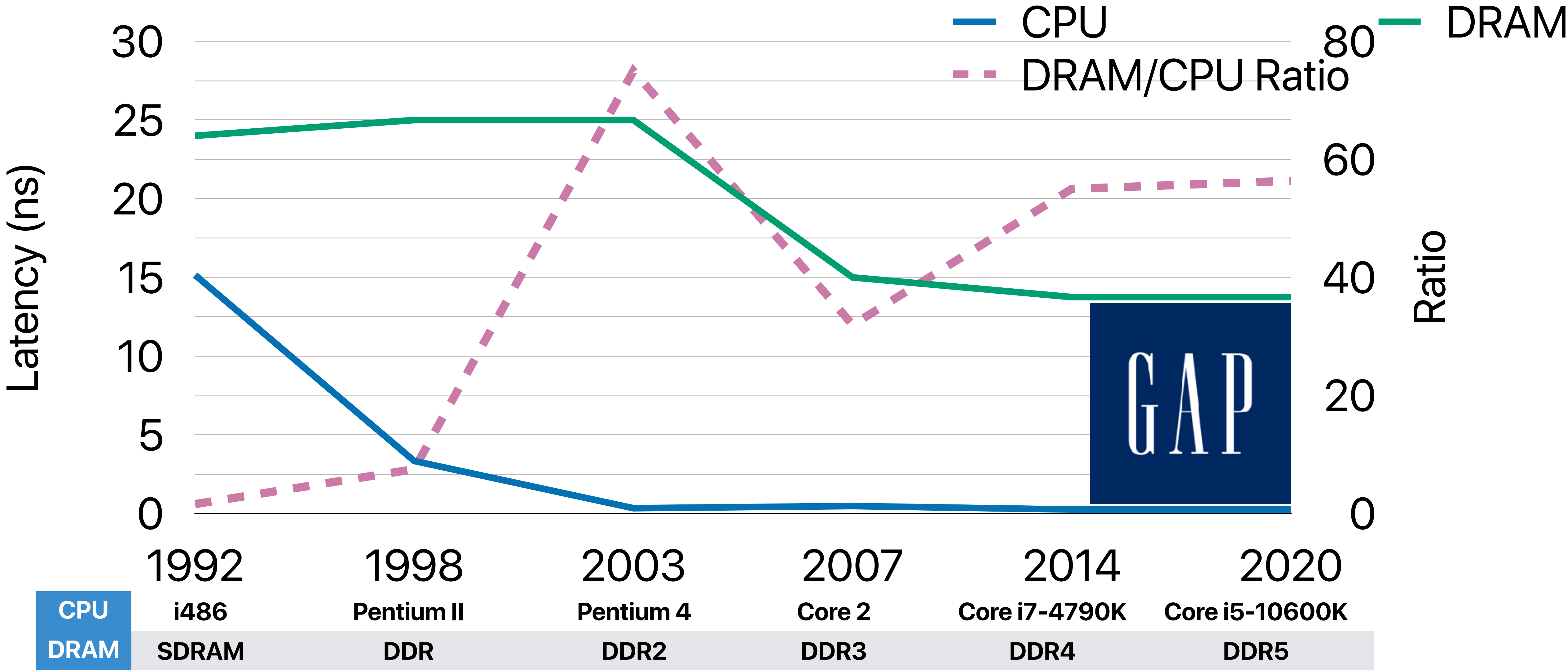
Memory hierarchy (2): the ABCs of our memory hierarchy

Hung-Wei Tseng

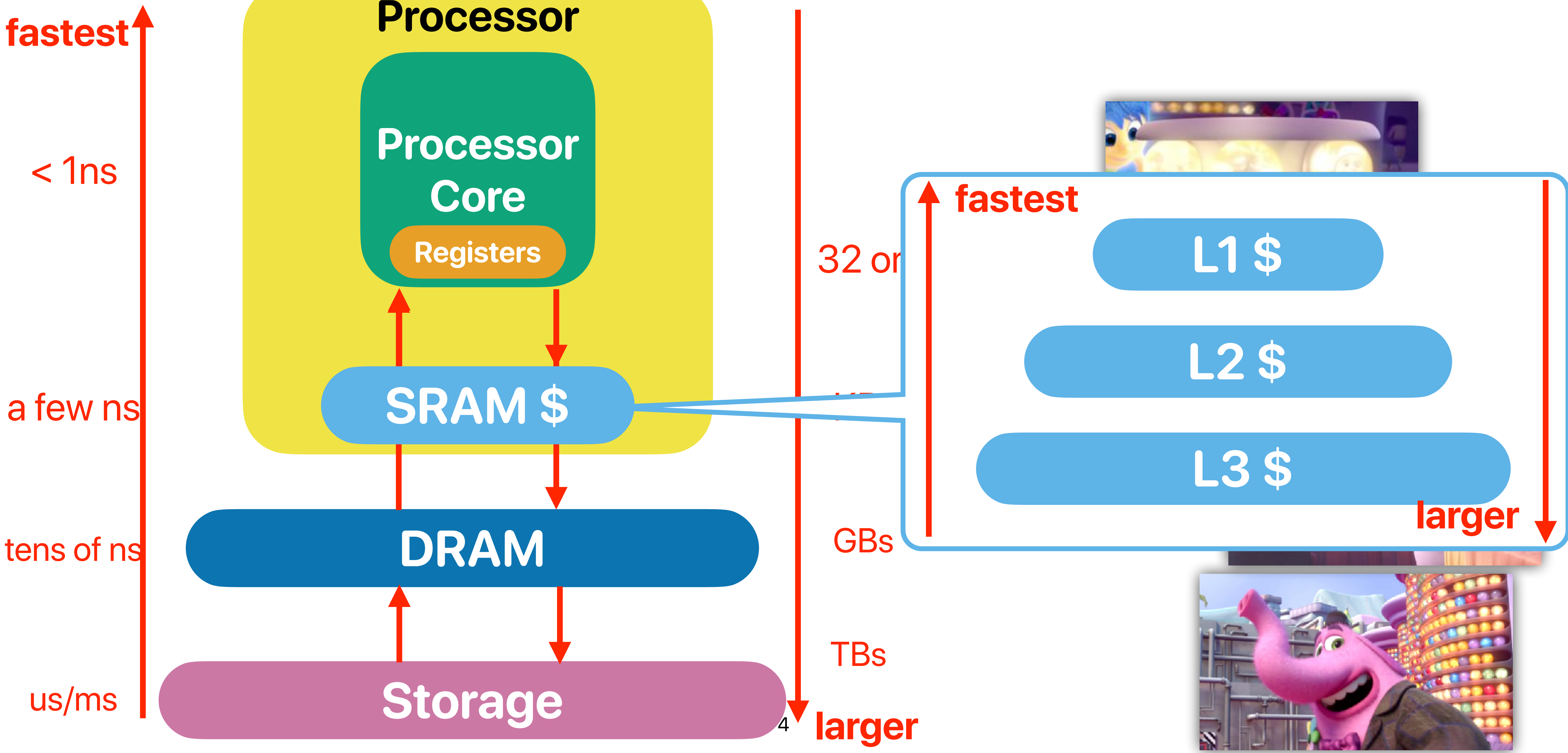
Recap: von Neumann architecture



The "latency" gap between CPU and DRAM

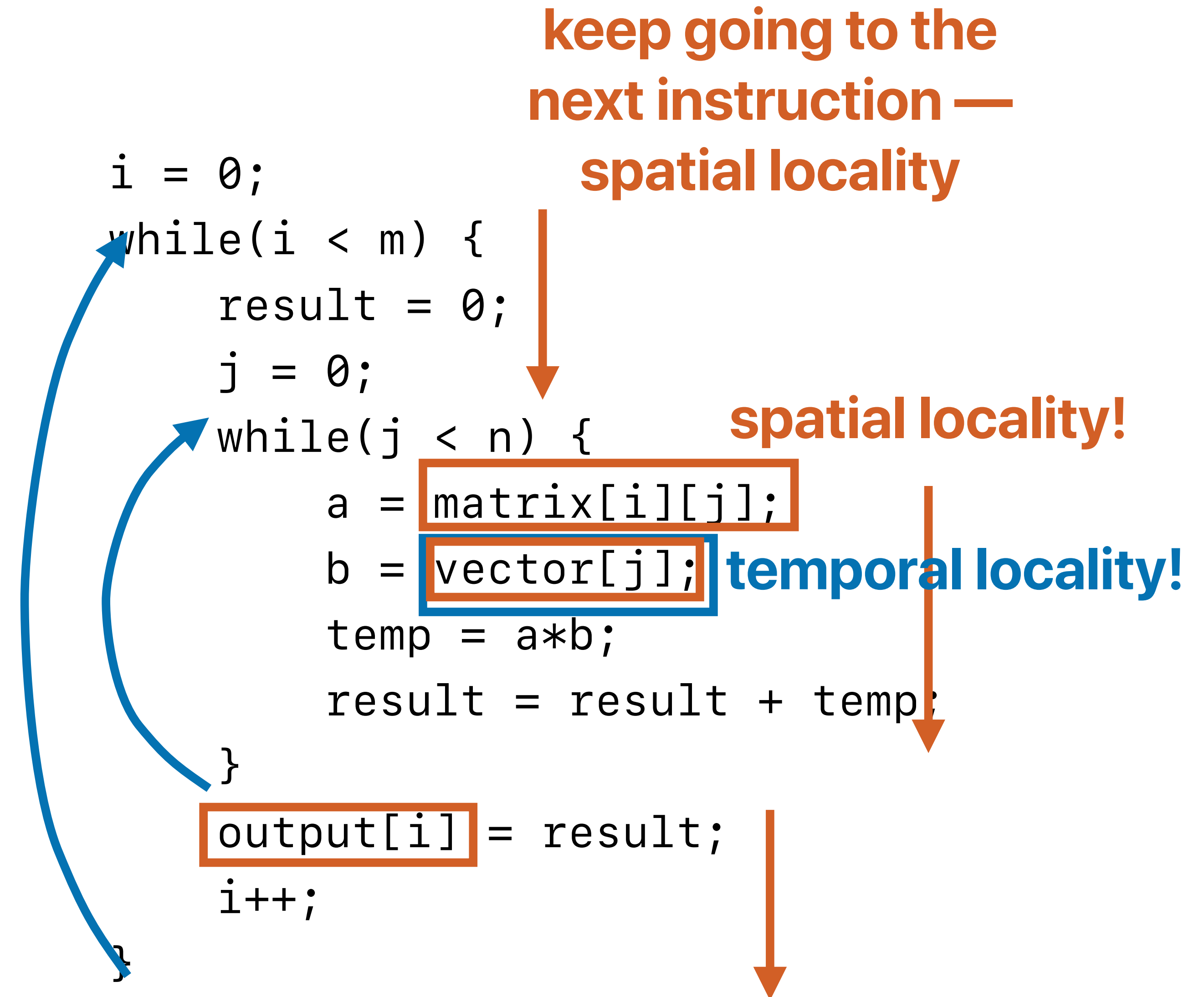
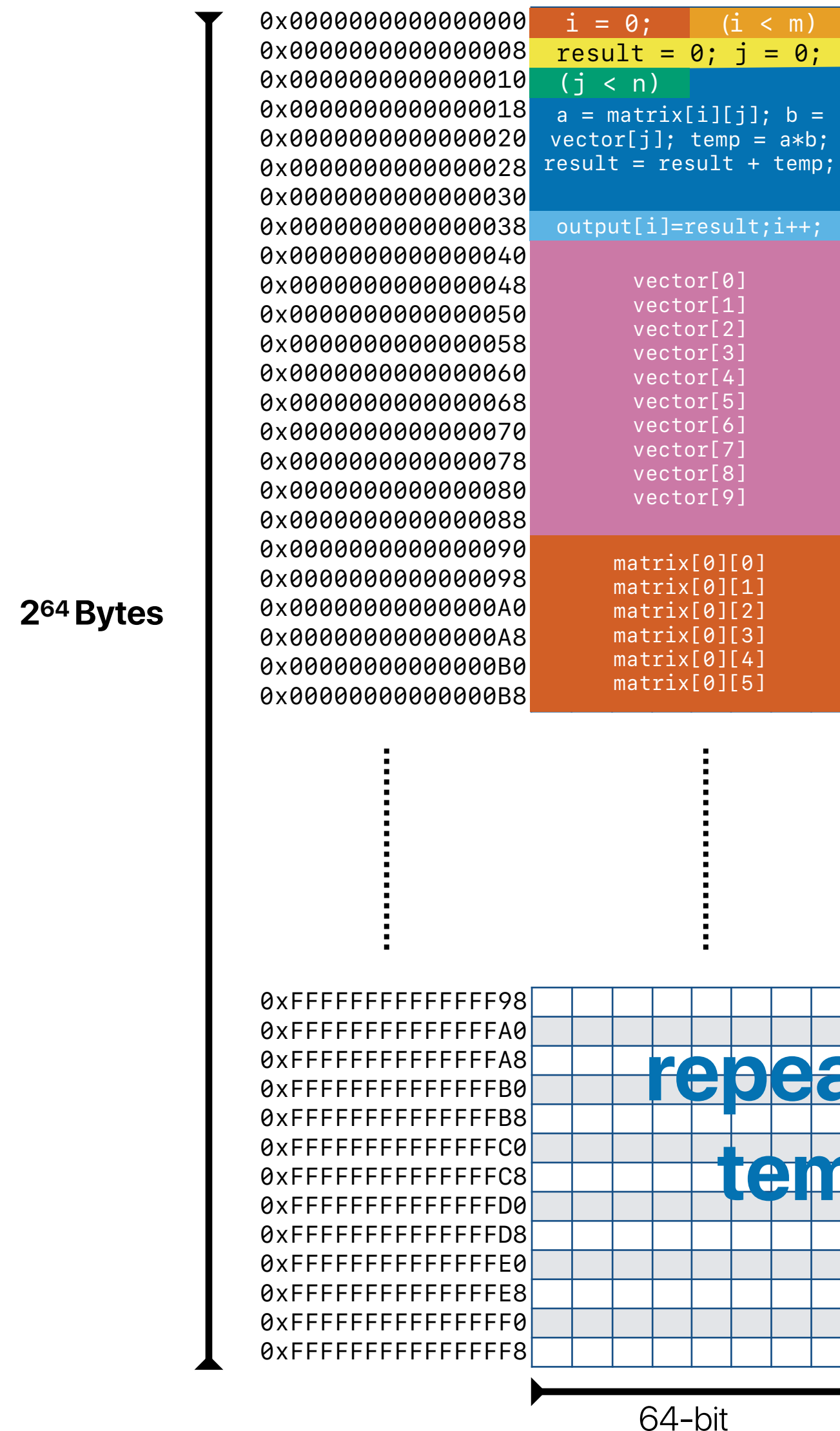


Memory Hierarchy



Locality in applications

Memory



Designing a hardware to exploit locality

- Spatial locality — application tends to visit nearby stuffs in the memory

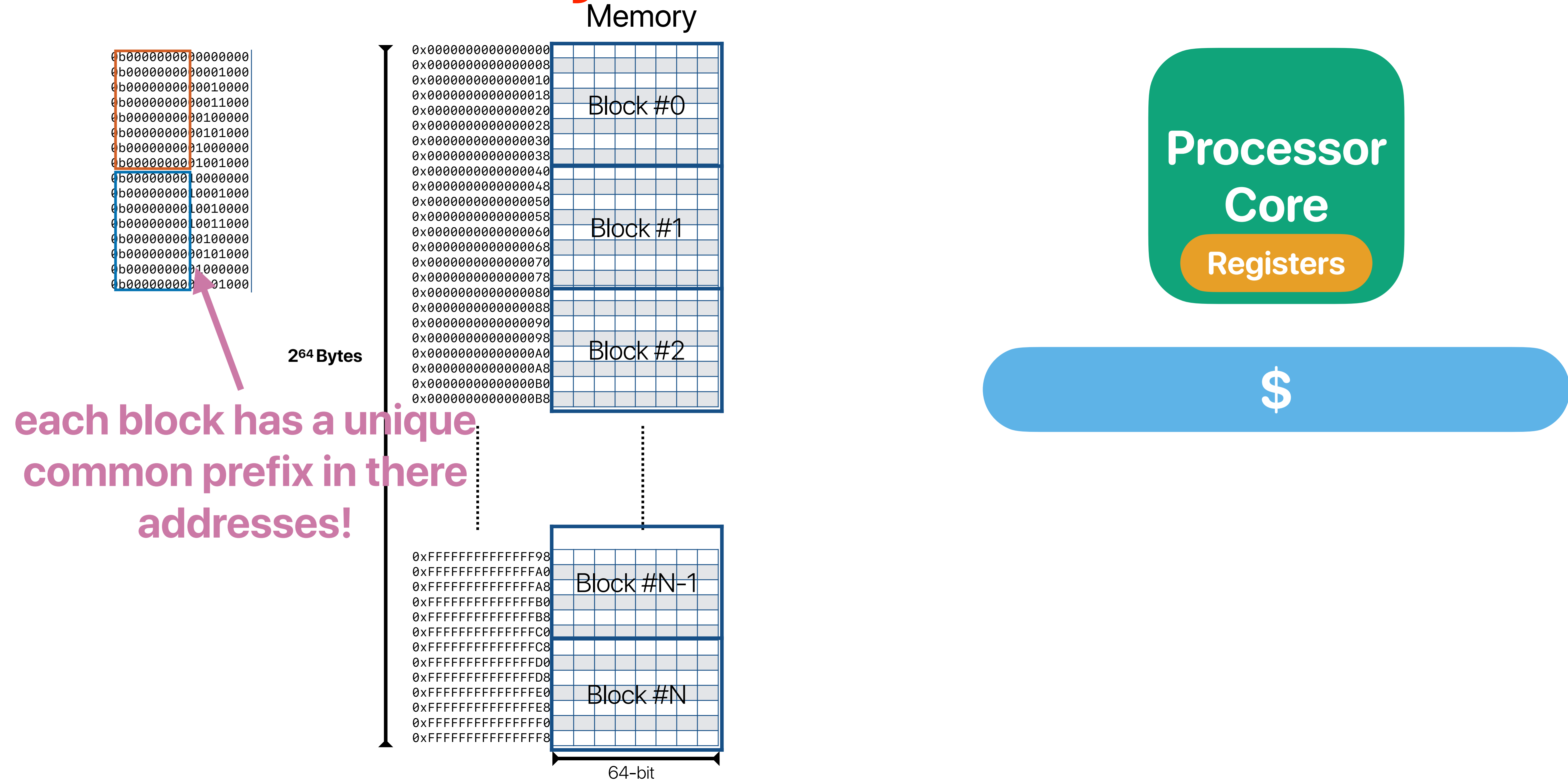
We need to "cache consecutive memory locations" every time — the cache should store a "block" of code/data

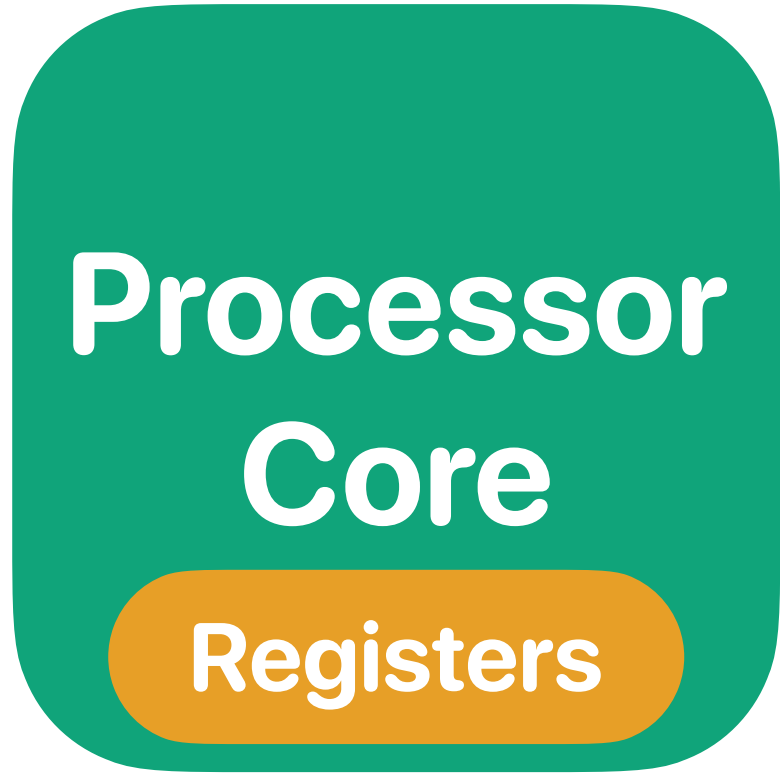
- Temporal locality — application revisit the same thing again and again

We need to "cache frequently used memory blocks"

- **Code** — loops, frequently invoked functions
 - Typically tens of static instructions — at most several KBs
- **Data** — program can read/write the same data many times (e.g., vectors in matrix-vector product)
 - **the cache should store a few blocks**
 - **the cache must be able to distinguish blocks**

Partition memory addresses into fix-sized chunks





How to tell who is there?

tag array

00000000000000000000000000000000
0x000000000x000000000x000000000x00000000
0123456789ABCDEF

the common address prefix in each block



0x000	This is CS 203:
0x001	Advanced Compute
0xF07	r Architecture!
0x100	This is CS 203:
0x310	Advanced Compute
0x450	r Architecture!
0x006	This is CS 203:
0x537	Advanced Compute
0x266	r Architecture!
0x307	This is CS 203:
0x265	Advanced Compute
0x80A	r Architecture!
0x620	This is CS 203:
0x630	Advanced Compute
0x705	r Architecture!
0x216	This is CS 203:

How to tell w



Tell if the block here can be used

Tell if the block here is modified

block offset
tag

1w 0x0008

1w 0x4048

0x404 not found,
go to lower-level memory

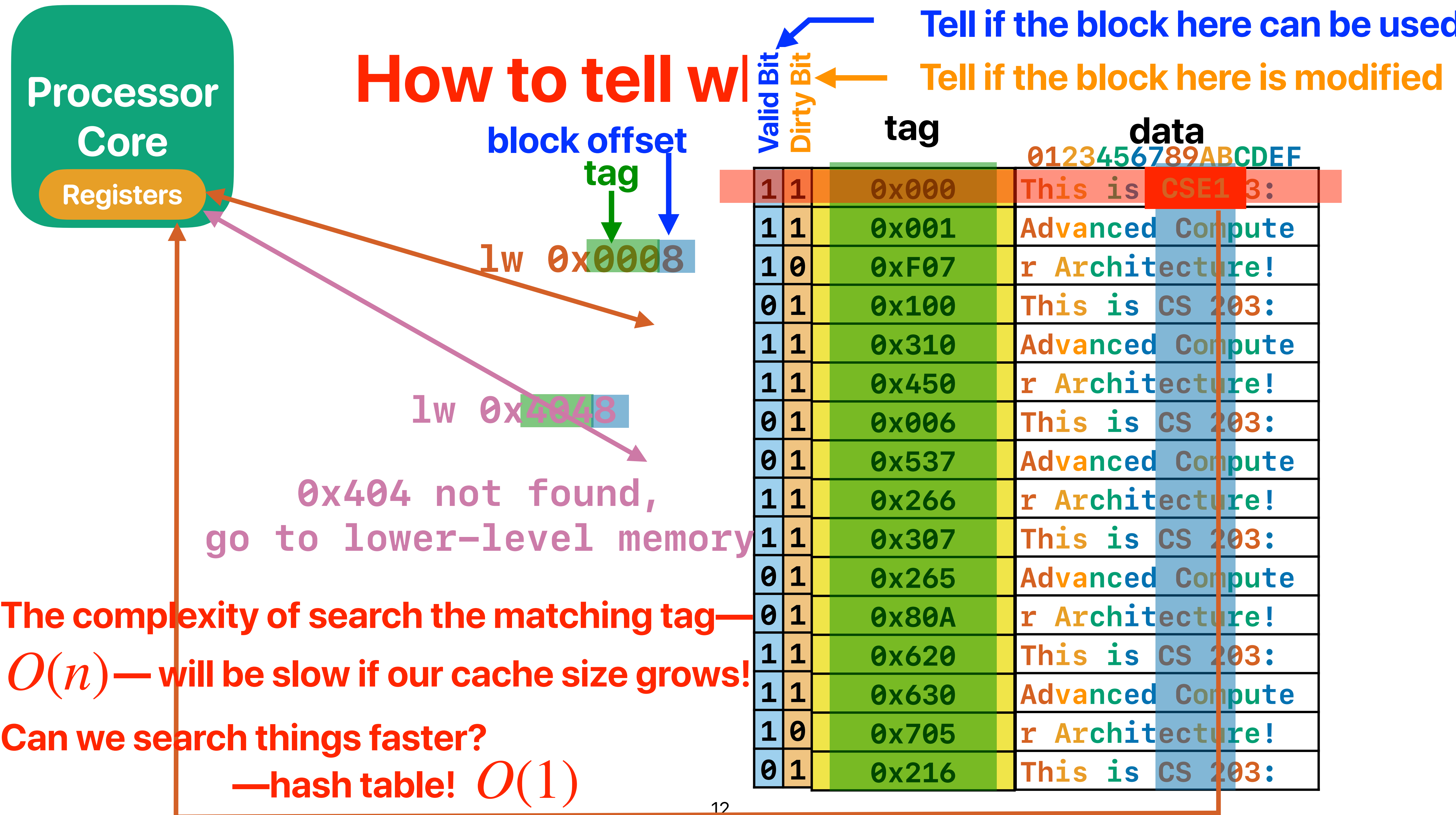
Valid Bit		Dirty Bit		tag	data
1	1				0123456789ABCDEF
1	1			0x000	This is CSE13:
1	1			0x001	Advanced Compute
1	0			0xF07	r Architecture!
0	1			0x100	This is CS 203:
1	1			0x310	Advanced Compute
1	1			0x450	r Architecture!
0	1			0x006	This is CS 203:
0	1			0x537	Advanced Compute
1	1			0x266	r Architecture!
1	1			0x307	This is CS 203:
0	1			0x265	Advanced Compute
0	1			0x80A	r Architecture!
1	1			0x620	This is CS 203:
1	1			0x630	Advanced Compute
1	0			0x705	r Architecture!
0	1			0x216	This is CS 203:

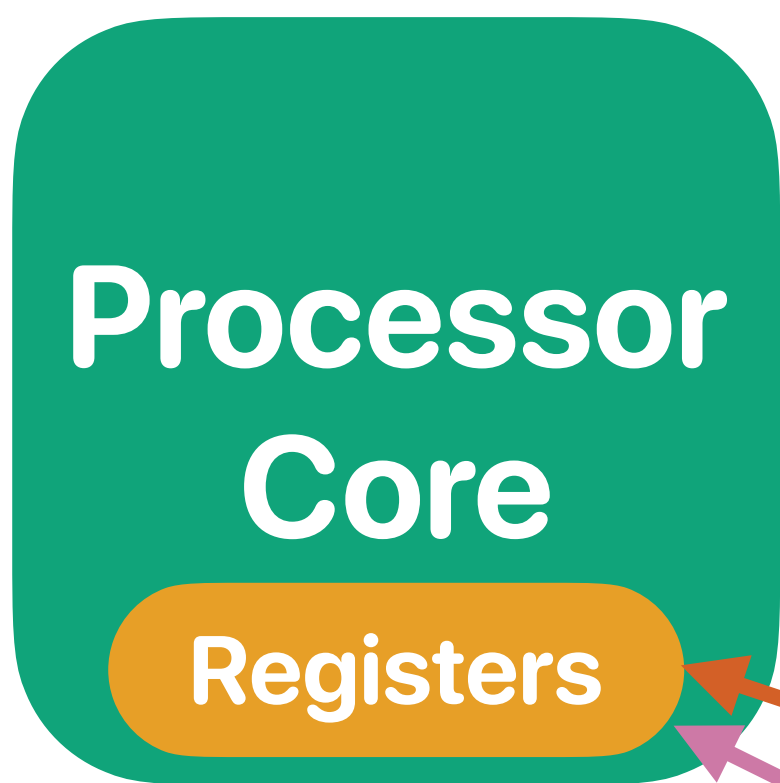
Outline

- Designing a cache that captures the predictability (cont.)
- The A, B, Cs of caches
- Estimating how cache friendly is our code

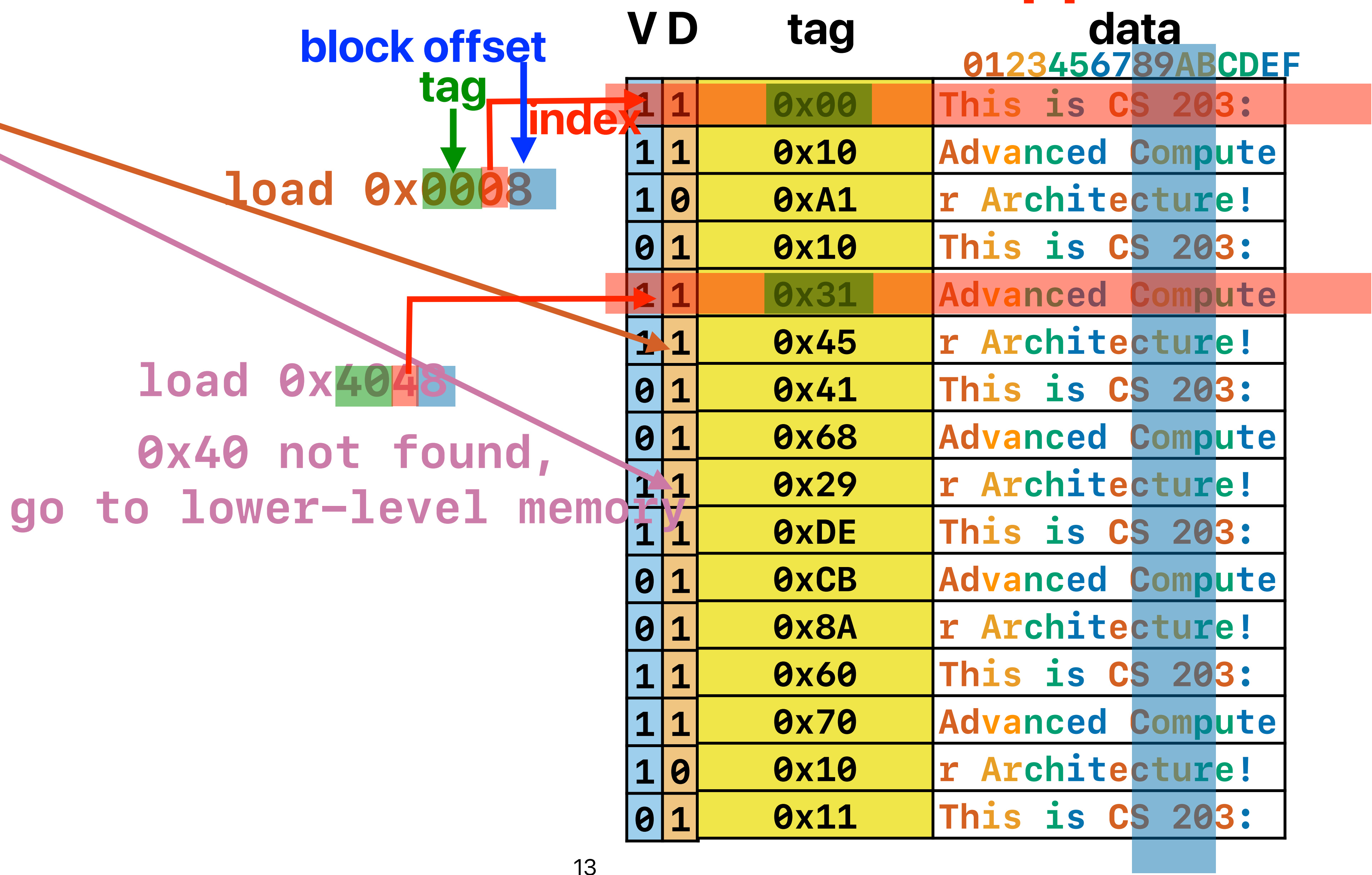
Architecting the cache (cont.)

How to tell w



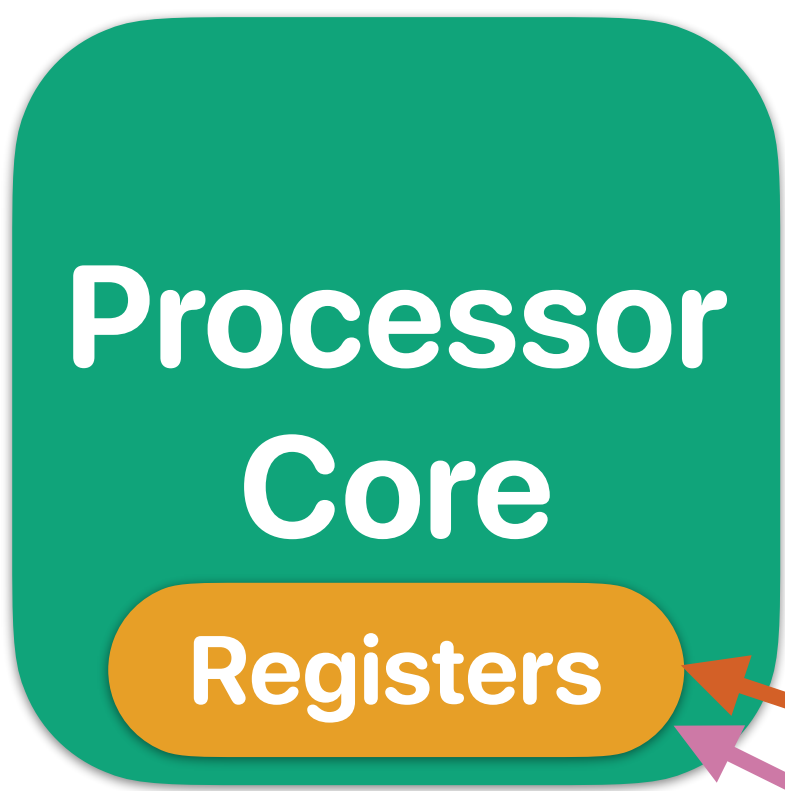


Hash-like structure — direct-mapped cache

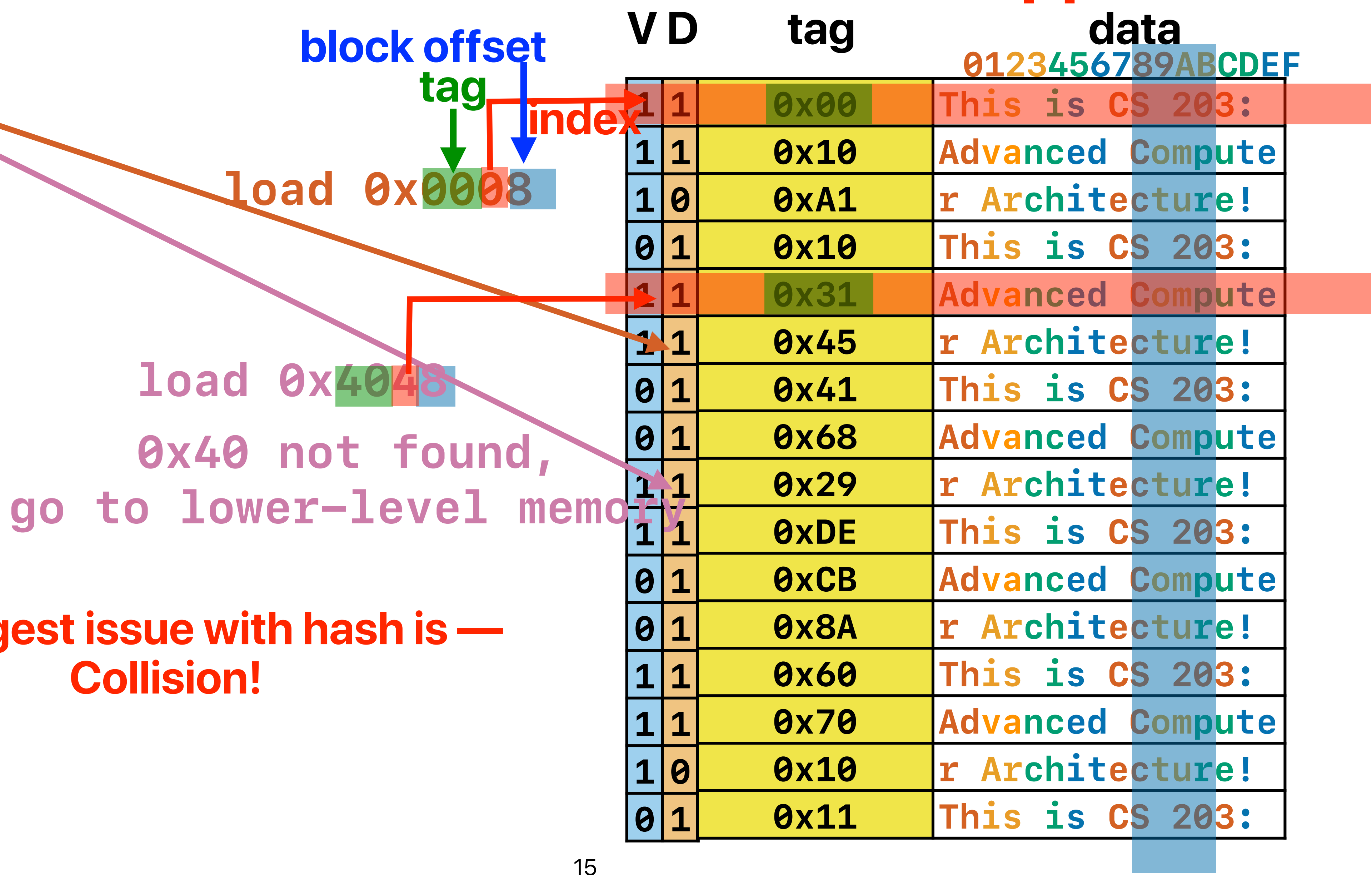


Take-aways: designing caches

- Basic cache structures —
 - Caching in granularity of a block to capture spatial locality
 - Caching multiple blocks to keep frequently used data — temporal locality
 - Tags to distinguish cached blocks
- Optimizing cache structures
 - Hash block into "sets" to reduce the search time



Hash-like structure — direct-mapped cache



The biggest issue with hash is — Collision!

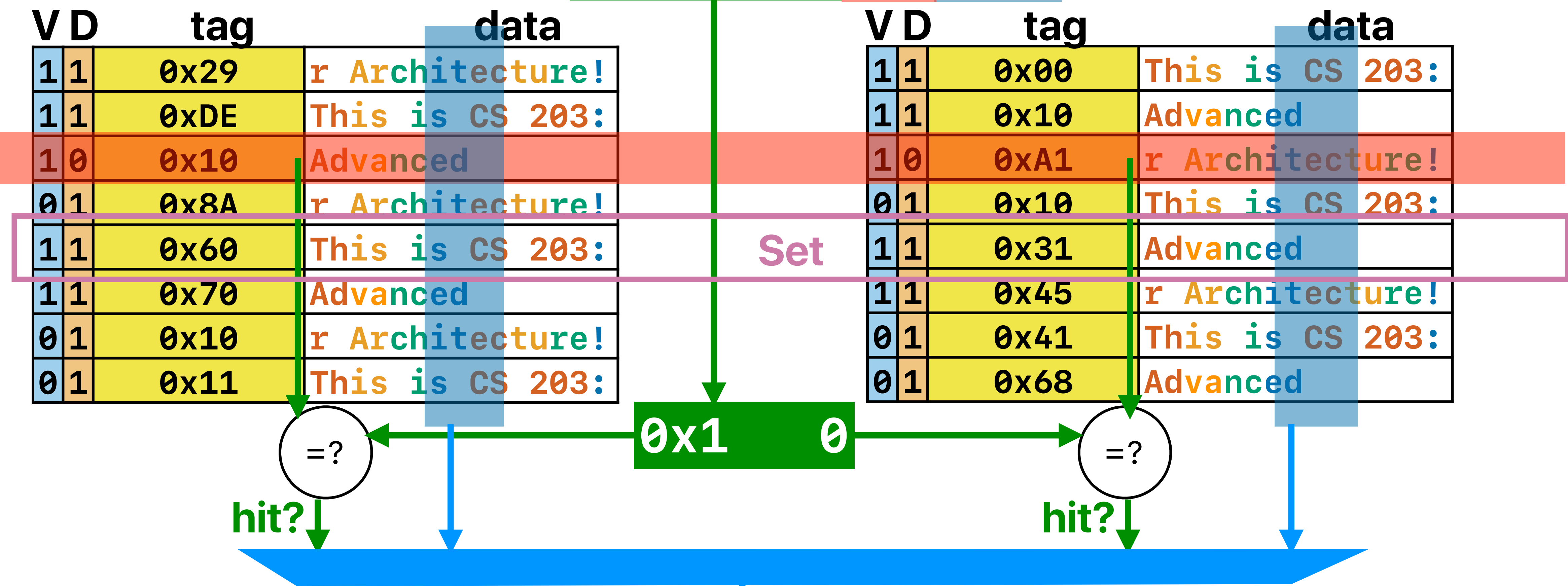
Way-associative cache

memory address: 0x0 8 2 4

set block

tag index offset

memory address: 0b00001000000100100



Blocksize == Linesize

```
/sys/devices/system/cpu/cpu0/cache/index0/physical_line_partition:1
/sys/devices/system/cpu/cpu0/cache/index0/number_of_sets:64
/sys/devices/system/cpu/cpu0/cache/index0/ways_of_associativity:12
/sys/devices/system/cpu/cpu0/cache/index0/id:0
/sys/devices/system/cpu/cpu0/cache/index0/shared_cpu_list:0-1
/sys/devices/system/cpu/cpu0/cache/index0/type:Data
/sys/devices/system/cpu/cpu0/cache/index0/size:48K
/sys/devices/system/cpu/cpu0/cache/index0/level:1
/sys/devices/system/cpu/cpu0/cache/index0/coherency_line_size:64
/sys/devices/system/cpu/cpu0/cache/index0/shared_cpu_map:000003
```

Blocksize == Linesize

```
/sys/devices/system/cpu/cpu0/cache/index0/physical_line_partition:1  
/sys/devices/system/cpu/cpu0/cache/index0/number of sets:64  
/sys/devices/system/cpu/cpu0/cache/index0/ways of associativity:12  
/sys/devices/system/cpu/cpu0/cache/index0/id:0  
/sys/devices/system/cpu/cpu0/cache/index0/shared_cpu_list:0-1  
/sys/devices/system/cpu/cpu0/cache/index0/type:Data  
/sys/devices/system/cpu/cpu0/cache/index0/size:48K  
/sys/devices/system/cpu/cpu0/cache/index0/level:1  
/sys/devices/system/cpu/cpu0/cache/index0/coherency_line_size:64  
/sys/devices/system/cpu/cpu0/cache/index0/shared_cpu_map:000003
```

Take-aways: designing caches

- Basic cache structures —
 - Caching in granularity of a block to capture spatial locality
 - Caching multiple blocks to keep frequently used data — temporal locality
 - Tags to distinguish cached blocks
- Hierarchical caching — data must be presented on the top level (L1) before the processor can use

Take-aways: designing caches

- Basic cache structures —
 - Caching in granularity of a block to capture spatial locality
 - Caching multiple blocks to keep frequently used data — temporal locality
 - Tags to distinguish cached blocks
- Optimizing cache structures
 - Hash block into "sets" to reduce the search time
 - Set-associativity to reduce the "collision" problem

Way-associative cache

memory address: $0x0$ 8 2 4

set block

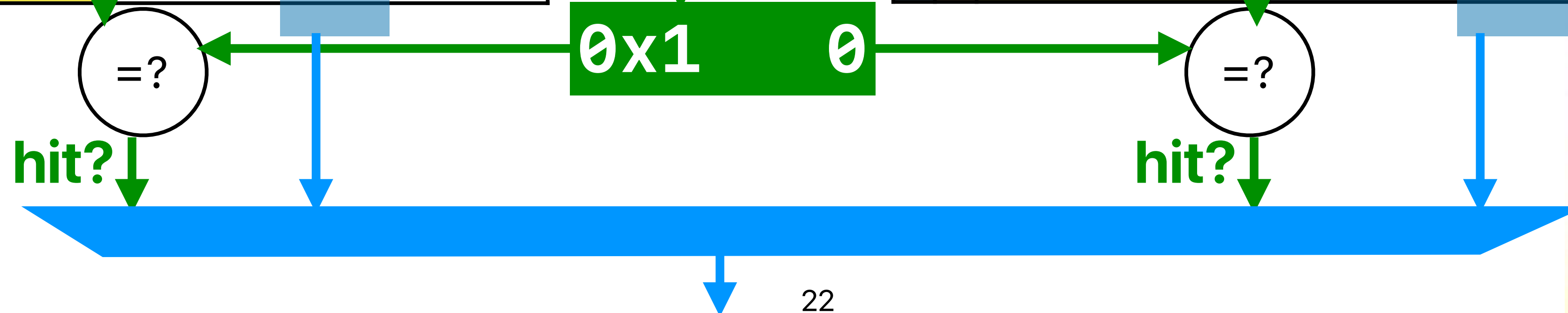
tag index offset

memory address: $0b00001000000100100$

V	D	tag	data
1	1	$0x29$	r Architecture!
1	1	$0xDE$	This is CS 203:
1	0	$0x10$	Advanced
0	1	$0x8A$	r Architecture!
1	1	$0x60$	This is CS 203:
1	1	$0x70$	Advanced
0	1	$0x10$	r Architecture!
0	1	$0x11$	This is CS 203:

V	D	tag	data
1	1	$0x00$	This is CS 203:
1	1	$0x10$	Advanced
1	0	$0xA1$	r Architecture!
0	1	$0x10$	This is CS 203:
1	1	$0x31$	Advanced
1	1	$0x45$	r Architecture!
0	1	$0x41$	This is CS 203:
0	1	$0x68$	Advanced

Set



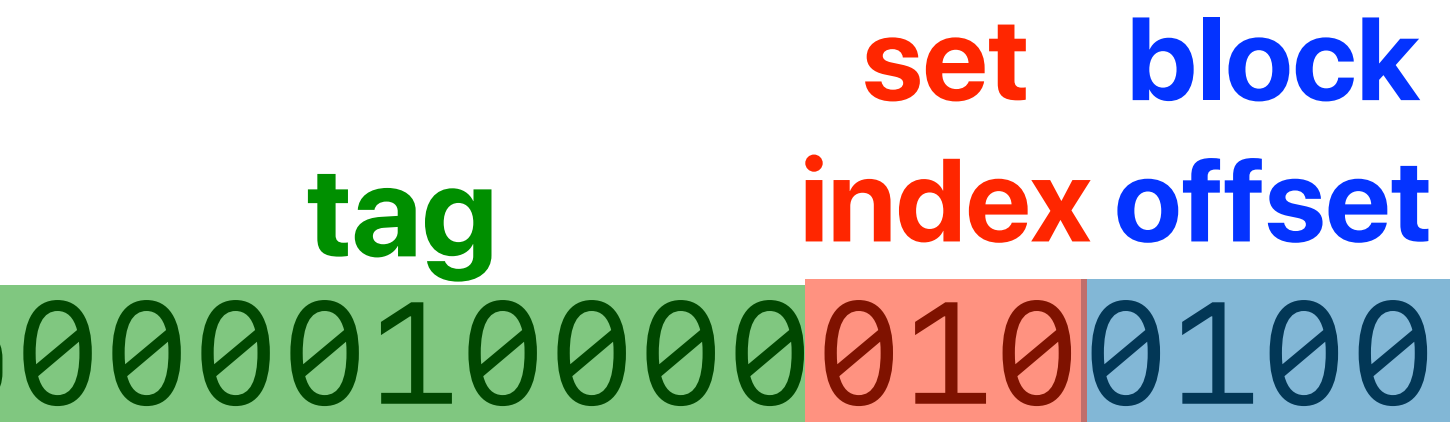
The A, B, Cs of your cache

$$C = ABS$$

- **C**: Capacity in data arrays
- **A**: Way-Associativity — how many blocks within a set
 - N-way: N blocks in a set, $A = N$
 - 1 for direct-mapped cache
- **B**: Block Size (Linesize)
 - How many bytes in a block
- **S**: Number of Sets:
 - A set contains blocks sharing the same index
 - 1 for fully associate cache



Corollary of $C = ABS$

memory address: 0b 

- number of bits in **block** offset — $\lg(\mathbf{B})$
- number of bits in **set** index: $\lg(\mathbf{S})$
- tag bits: $\text{address_length} - \lg(\mathbf{S}) - \lg(\mathbf{B})$
 - address_length is N bits for N-bit machines (e.g., 64-bit for 64-bit machines)
- $(\text{address} / \text{block_size}) \% \mathbf{S} = \text{set index}$



NVIDIA Tegra Orin

- L1 data (D-L1) cache configuration of NVIDIA Tegra Orin
 - Size 64KB, 4-way set associativity, 64B block
 - Assume 64-bit memory address

Which of the following is correct?

- A. Tag is 49 bits
- B. Index is 8 bits
- C. Offset is 7 bits
- D. The cache has 1024 sets
- E. None of the above

A screenshot of a Pollev poll interface. It shows five horizontal input fields, each preceded by a letter from A to E. The fields are currently empty, indicating that no answers have been submitted yet. The interface is clean and modern, with a light blue header and a white background for the poll options.

NVIDIA Tegra Orin

- L1 data (D-L1) cache configuration of NVIDIA Tegra Orin
 - Size 64KB, 4-way set associativity, 64B block
 - Assume 64-bit memory address

Which of the following is correct?

A. Tag is 49 bits

B. Index is 8 bits

C. Offset is 7 bits

D. The cache has 1024 sets

E. None of the above

$$C = A \times B \times S$$

$$64 \times 1024 = 4 \times 64 \times S$$

$$S = 256$$

$$Offset = \log_2(64) = 6$$

$$Index = \log_2(256) = 8$$

$$Tag = 64 - 8 - 6 = 50$$



intel Core i7 14700K

- L1 data (D-L1) cache configuration of Core i7 14700K's P-Core
 - Size 48KB, 12-way set associativity, 64B block
 - Assume 64-bit memory address
 - Which of the following is **NOT** correct?
 - A. Tag is 52 bits
 - B. Index is 6 bits
 - C. Offset is 6 bits
 - D. The cache has 128 sets
 - E. All of the above are correct

A

B

C

D

E

intel Core i7 14700K

- L1 data (D-L1) cache configuration of Core i7 14700K's P-Core
 - Size 48KB, 12-way set associativity, 64B block
 - Assume 64-bit memory address
 - Which of the following is **NOT** correct?

A. Tag is 52 bits

B. Index is 6 bits

C. Offset is 6 bits

D. The cache has 128 sets

E. All of the above are correct

$$C = A \times B \times S$$

$$48 \times 1024 = 12 \times 64 \times S$$

$$S = 64$$

$$Offset = \log_2(64) = 6$$

$$Index = \log_2(12) = 3.58$$

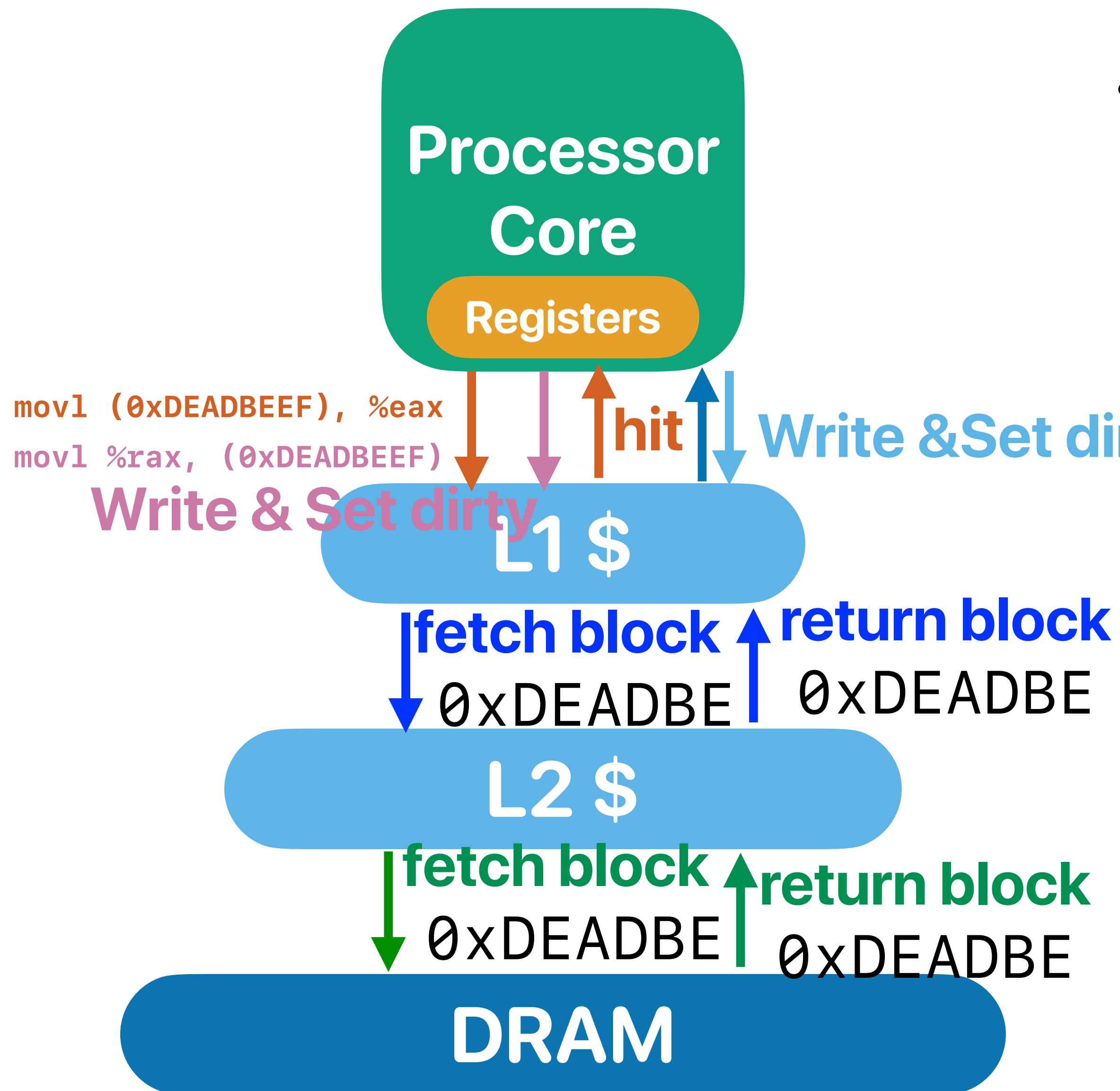
$$Tag = 64 - 6 - 6 = 52$$

Take-aways: designing caches

- Basic cache structures —
 - Caching in granularity of a block to capture spatial locality
 - Caching multiple blocks to keep frequently used data — temporal locality
 - Tags to distinguish cached blocks
- Hierarchical caching — data must be presented on the top level (L1) before the processor can use
- Optimizing cache structures
 - Hash block into “sets” to reduce the search time
 - Set-associativity to reduce the “collision” problem
- $C = A B S$
 - C: capacity
 - A: Associativity
 - S: Number of sets
 - $\lg(S)$: Number of bits in set index
 - $\lg(B)$: Number of bits in block offset

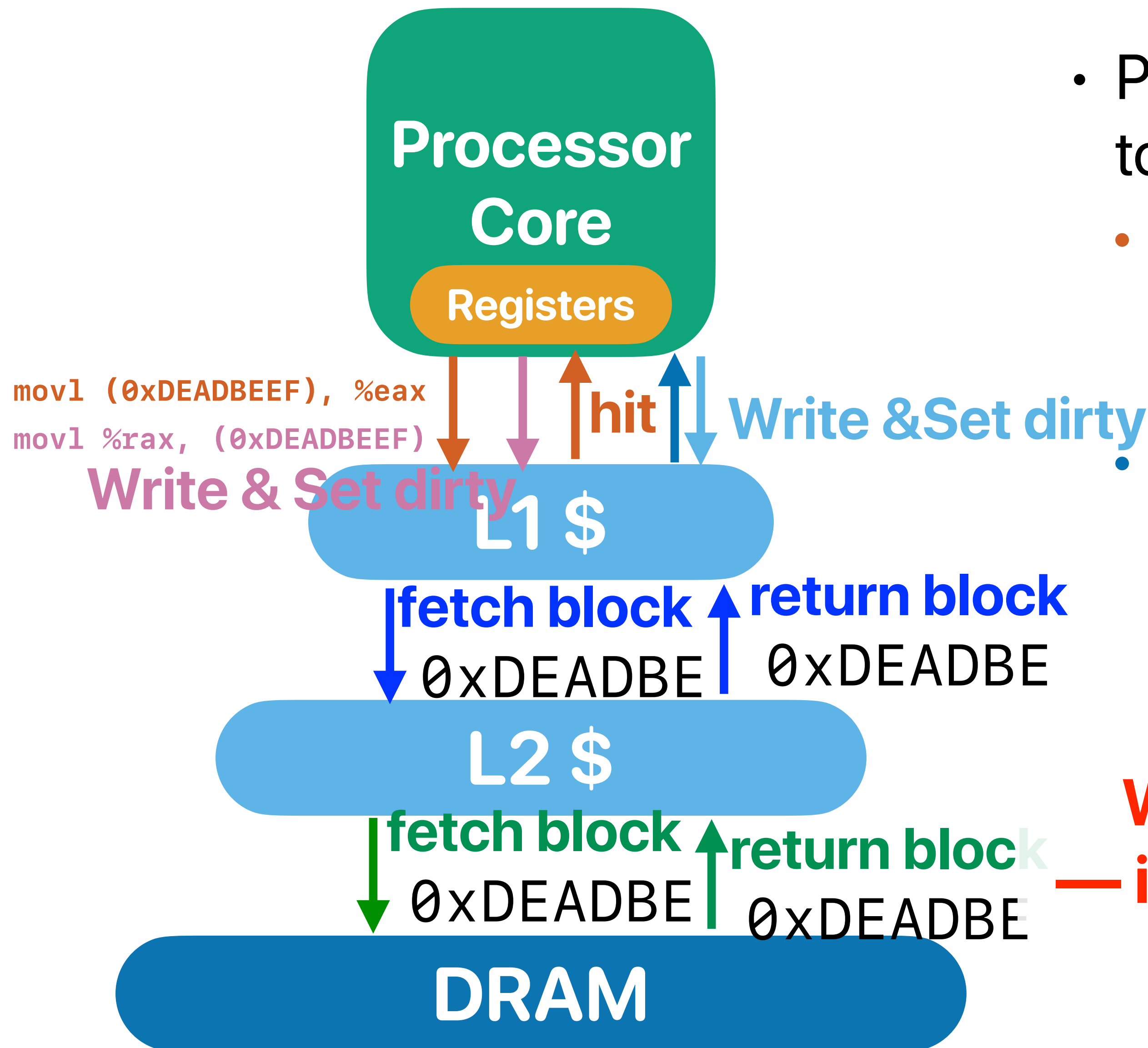
**Put everything all together:
How cache interacts with CPU**

Processor/cache interaction



- Processor sends memory access request to L1-\$
 - **if hit & it's a read**
 - **Read:** return data
 - **Write:** Update "ONLY" in L1 and set DIRTY
 - **Why don't we write to L2?** — Too slow
 - **if miss**
 - Fetch the requesting block from lower-level memory hierarchy and place in the cache
 - **Present the write "ONLY" in L1 and set DIRTY**
- What if we run out of \$ blocks?**

Processor/cache interaction

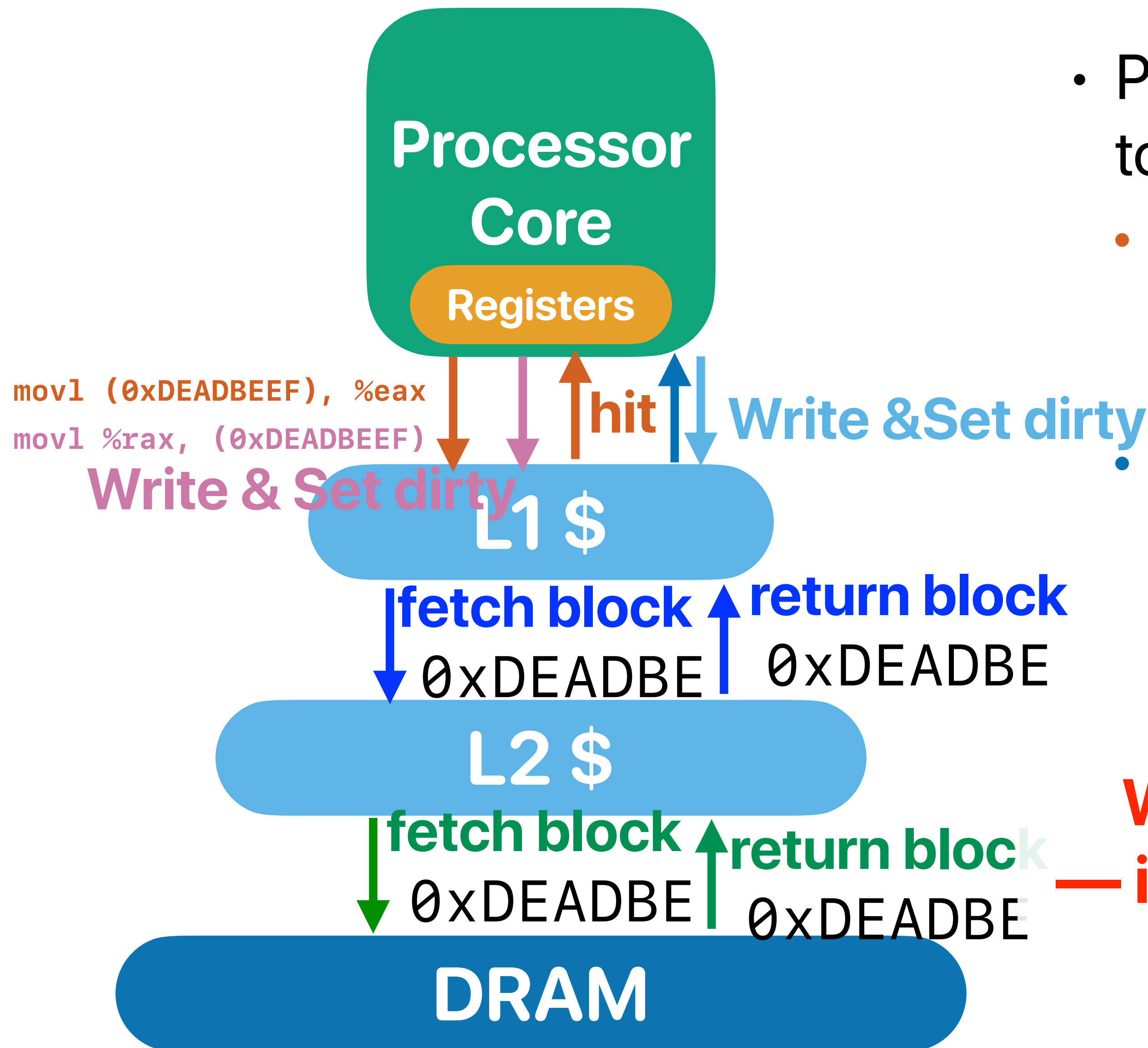


- Processor sends memory access request to L1-\$
- if hit & it's a read
 - Read: return data
- Write: Update "ONLY" in L1 and set DIRTY
- if miss
 - If there an empty block — place the data there
 - If NOT (most frequent case) — select a **victim block**
 - Least Recently Used (LRU) policy

What if the victim block is modified?
— ignoring the update is not acceptable!

- Present the write "ONLY" in L1 and set DIRTY

Processor/cache interaction

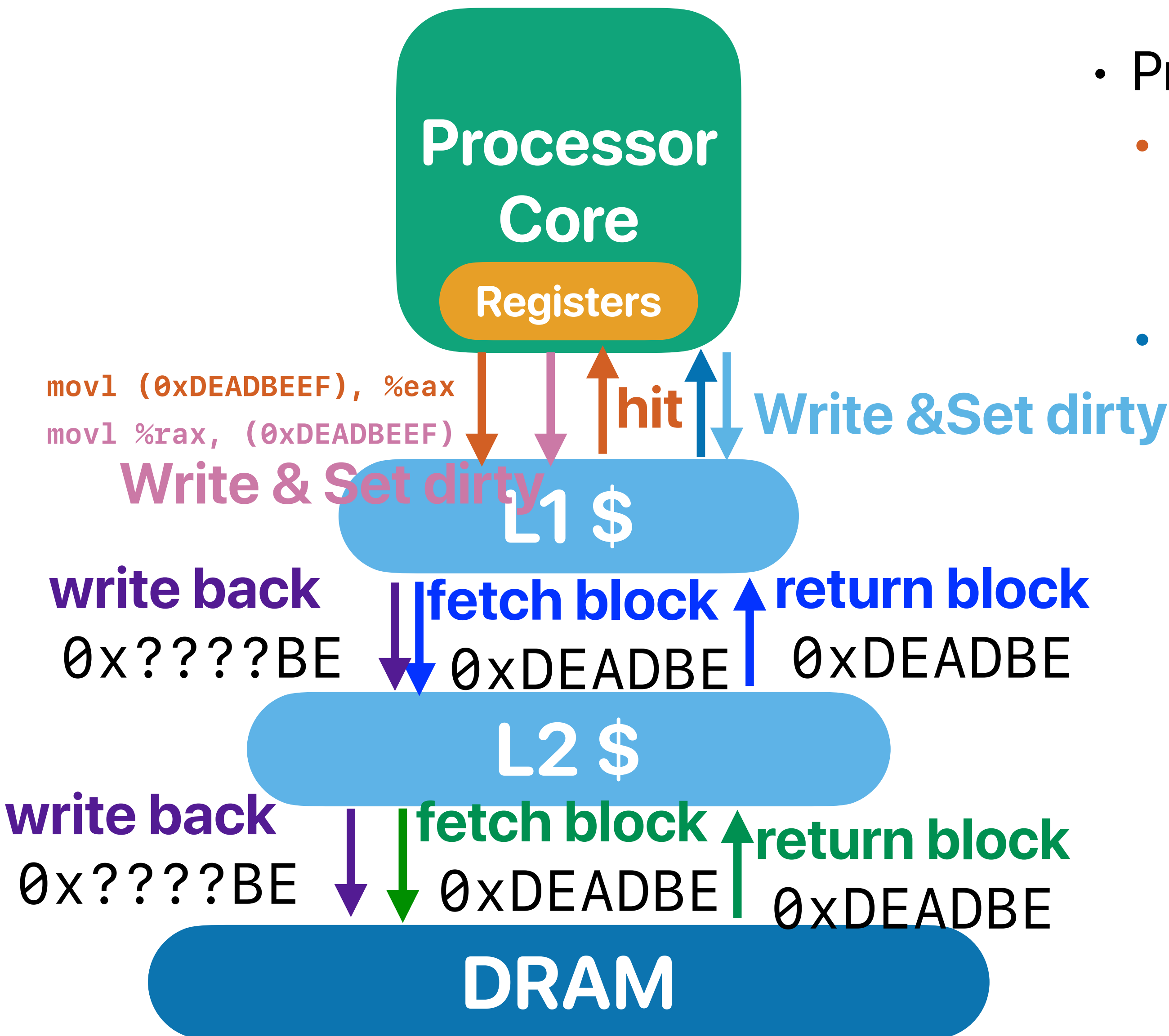


- Processor sends memory access request to L1-\$
- **if hit & it's a read**
 - **Read:** return data
- **Write: Update "ONLY" in L1 and set DIRTY**
- **if miss**
 - If there an empty block — place the data there
 - If NOT (most frequent case) — select a **victim block**
 - Least Recently Used (LRU) policy

What if the victim block is modified?
— ignoring the update is not acceptable!

- **Present the write "ONLY" in L1 and set DIRTY**

Processor/cache interaction



- Processor sends memory access request to L1-\$
 - **if hit & it's a read**
 - **Read:** return data
 - **Write:** Update "ONLY" in L1 and set DIRTY
 - **if miss**
 - If there an empty block — place the data there
 - If NOT (most frequent case) — select a **victim block**
 - Least Recently Used (LRU) policy
 - If the victim block is "dirty" & "valid"
 - **Write back** the block to lower-level memory hierarchy
 - If write-back or fetching causes any miss, repeat the same process
 - Fetch the requesting block from lower-level memory hierarchy and place in the cache
 - **Present the write "ONLY" in L1 and set DIRTY**

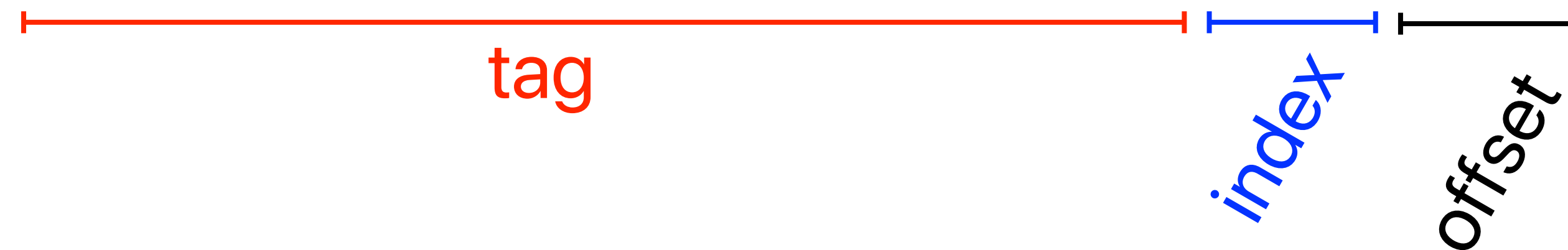
Simulate the cache!

Simulate a direct-mapped cache

- A direct mapped (1-way) cache with 256 bytes total capacity, a block size of 16 bytes

- # of blocks = $\frac{256}{16} = 16$
- $\lg(16) = 4$: 4 bits are used for the index
- $\lg(16) = 4$: 4 bits are used for the byte offset
- The tag is $64 - (4 + 4) = 56$ bits

- For example: 0x 8 0 0 0 0 0 8 0
= 0b1000 0000 0000 0000 0000 0000 1000 0000



Matrix vector revisited

```
for(uint64_t i = 0; i < m; i++) {  
    result = 0;  
    for(uint64_t j = 0; j < n; j++) {  
        result += matrix[i][j]*vector[j];  
    }  
    output[i] = result;  
}
```

Matrix vector revisited

```
for(uint64_t i = 0; i < m; i++) {
    result = 0;
    for(uint64_t j = 0; j < n; j++) {
        result += matrix[i][j]*vector[j];
    }
    output[i] = result;
}
```

tag index

tag index

Address (Hex)		Address (Binary)	
&a[0][0]	0x558FE0A1D330	0b10101011000111111100000101000011101001100110000	
&b[0]	0x558FE0A1DC30	0b10101011000111111100000101000011101110000110000	
&a[0][1]	0x558FE0A1D338	0b10101011000111111100000101000011101001100111000	
&b[1]	0x558FE0A1DC38	0b10101011000111111100000101000011101110000111000	
&a[0][2]	0x558FE0A1D340	0b10101011000111111100000101000011101001101000000	
&b[2]	0x558FE0A1DC40	0b10101011000111111100000101000011101110001000000	
&a[0][3]	0x558FE0A1D348	0b10101011000111111100000101000011101001101001000	
&b[3]	0x558FE0A1DC48	0b101010110001111111000001010000111011100001001000	
&a[0][4]	0x558FE0A1D350	0b10101011000111111100000101000011101001101010000	
&b[4]	0x558FE0A1DC50	0b101010110001111111000001010000111011100001010000	
&a[0][5]	0x558FE0A1D358	0b10101011000111111100000101000011101001101011000	
&b[5]	0x558FE0A1DC58	0b101010110001111111000001010000111011100001011000	
&a[0][6]	0x558FE0A1D360	0b10101011000111111100000101000011101001101100000	
&b[6]	0x558FE0A1DC60	0b101010110001111111000001010000111011100001100000	
&a[0][7]	0x558FE0A1D368	0b10101011000111111100000101000011101001101101000	
&b[7]	0x558FE0A1DC68	0b101010110001111111000001010000111011100001101000	
&a[0][8]	0x558FE0A1D370	0b10101011000111111100000101000011101001101110000	
&b[8]	0x558FE0A1DC70	0b101010110001111111000001010000111011100001110000	
&a[0][9]	0x558FE0A1D378	0b10101011000111111100000101000011101001101111000	
&b[9]	0x558FE0A1DC78	0b101010110001111111000001010000111011100001111000	

Simulate a direct-mapped cache

tag index

	V	D	Tag	Data
0	0	0		
1	0	0		
2	0	0		
3	1	0	0x558FE0A1DC	b[0], b[1]
4	1	0	0x558FE0A1DC	b[2], b[3]
5	0	0		
6	0	0		
7	0	0		
8	0	0		
9	0	0		
10	0	0		
11	0	0		
12	0	0		
13	0	0		
14	0	0		
15	0	0		

This cache doesn't work!!!
— collisions!

Address (Hex)		
&a[0][0]	0x558FE0A1D330	miss
&b[0]	0x558FE0A1DC30	miss
&a[0][1]	0x558FE0A1D338	miss
&b[1]	0x558FE0A1DC38	miss
&a[0][2]	0x558FE0A1D340	miss
&b[2]	0x558FE0A1DC40	miss
&a[0][3]	0x558FE0A1D348	miss
&b[3]	0x558FE0A1DC48	miss
&a[0][4]	0x558FE0A1D350	miss
&b[4]	0x558FE0A1DC50	miss
&a[0][5]	0x558FE0A1D358	miss
&b[5]	0x558FE0A1DC58	miss
&a[0][6]	0x558FE0A1D360	miss
&b[6]	0x558FE0A1DC60	miss
&a[0][7]	0x558FE0A1D368	miss
&b[7]	0x558FE0A1DC68	miss
&a[0][8]	0x558FE0A1D370	miss
&b[8]	0x558FE0A1DC70	miss
&a[0][9]	0x558FE0A1D378	
&b[9]	0x558FE0A1DC78	

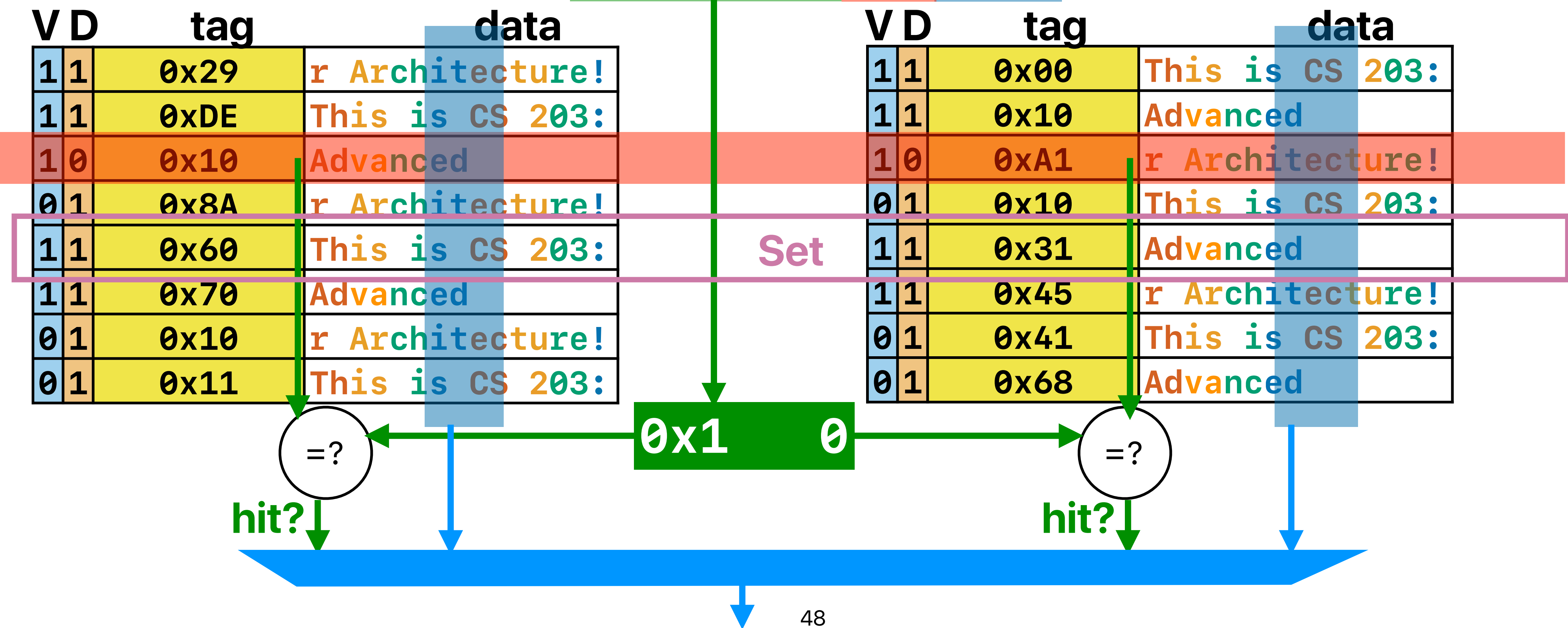
Way-associative cache

memory address: 0x0 8 2 4

set block

tag index offset

memory address: 0b00001000000100100



Now, 2-way, same-sized cache

- A 2-way cache with 256 bytes total capacity, a block size of 16 bytes

- # of blocks = $\frac{256}{16} = 16$
- # of sets = $\frac{16}{2} = 8$ (2-way: 2 blocks in a set)

- $\lg(8) = 3$: 3 bits are used for the index
- $\lg(16) = 4$: 4 bits are used for the byte offset
- The tag is $64 - (4 + 4) = 56$ bits

- For example: 0x 8 0 0 0 0 0 0 8 0
= 0b1000 0000 0000 0000 0000 0000 1000 0000



Matrix vector revisited

```
for(uint64_t i = 0; i < m; i++) {
    result = 0;
    for(uint64_t j = 0; j < n; j++) {
        result += matrix[i][j]*vector[j];
    }
    output[i] = result;
}
```

	tag index		tag index	
	Address (Hex)		Address (Binary)	
&a[0][0]	0x558FE0A1D330	0b	10101011000111111100000101000011101001100110000	
&b[0]	0x558FE0A1DC30	0b	10101011000111111100000101000011101110000110000	
&a[0][1]	0x558FE0A1D338	0b	10101011000111111100000101000011101001100111000	
&b[1]	0x558FE0A1DC38	0b	10101011000111111100000101000011101110000111000	
&a[0][2]	0x558FE0A1D340	0b	10101011000111111100000101000011101001101000000	
&b[2]	0x558FE0A1DC40	0b	10101011000111111100000101000011101110001000000	
&a[0][3]	0x558FE0A1D348	0b	10101011000111111100000101000011101001101001000	
&b[3]	0x558FE0A1DC48	0b	10101011000111111100000101000011101110001001000	
&a[0][4]	0x558FE0A1D350	0b	10101011000111111100000101000011101001101010000	
&b[4]	0x558FE0A1DC50	0b	10101011000111111100000101000011101110001010000	
&a[0][5]	0x558FE0A1D358	0b	10101011000111111100000101000011101001101011000	
&b[5]	0x558FE0A1DC58	0b	10101011000111111100000101000011101110001011000	
&a[0][6]	0x558FE0A1D360	0b	10101011000111111100000101000011101001101100000	
&b[6]	0x558FE0A1DC60	0b	10101011000111111100000101000011101110001100000	
&a[0][7]	0x558FE0A1D368	0b	10101011000111111100000101000011101001101101000	
&b[7]	0x558FE0A1DC68	0b	10101011000111111100000101000011101110001101000	
&a[0][8]	0x558FE0A1D370	0b	10101011000111111100000101000011101001101110000	
&b[8]	0x558FE0A1DC70	0b	10101011000111111100000101000011101110001110000	
&a[0][9]	0x558FE0A1D378	0b	10101011000111111100000101000011101001101111000	
&b[9]	0x558FE0A1DC78	0b	10101011000111111100000101000011101110001111000	

Simulate a 2-way cache

V	D	Tag	Data	V	D	Tag	Data
0	0			0	0		
0	0			0	0		
0	0			0	0		
1	0	0xAB1FC143A6	a[0][0], a[0][1]	1	0	0xAB1FC143B8	b[0], b[1]
1	0	0xAB1FC143A6	a[0][2], a[0][3]	1	0	0xAB1FC143B8	b[2], b[3]
0	0			0	0		
0	0			0	0		
0	0			0	0		

	Address (Hex)	Tag	Index	
&a[0][0]	0x558FE0A1D330	0xAB1FC143A6	0x3	miss
&b[0]	0x558FE0A1DC30	0xAB1FC143B8	0x3	miss
&a[0][1]	0x558FE0A1D338	0xAB1FC143A6	0x3	hit
&b[1]	0x558FE0A1DC38	0xAB1FC143B8	0x3	hit
&a[0][2]	0x558FE0A1D340	0xAB1FC143A6	0x4	miss
&b[2]	0x558FE0A1DC40	0xAB1FC143B8	0x4	miss
&a[0][3]	0x558FE0A1D348	0xAB1FC143A6	0x4	hit
&b[3]	0x558FE0A1DC48	0xAB1FC143B8	0x4	hit
&a[0][4]	0x558FE0A1D350	0xAB1FC143A6	0x5	miss
&b[4]	0x558FE0A1DC50	0xAB1FC143B8	0x5	miss
&a[0][5]	0x558FE0A1D358	0xAB1FC143A6	0x5	hit
&b[5]	0x558FE0A1DC58	0xAB1FC143B8	0x5	hit
&a[0][6]	0x558FE0A1D360	0xAB1FC143A6	0x6	miss
&b[6]	0x558FE0A1DC60	0xAB1FC143B8	0x6	miss
&a[0][7]	0x558FE0A1D368	0xAB1FC143A6	0x6	hit
&b[7]	0x558FE0A1DC68	0xAB1FC143B8	0x6	hit
&a[0][8]	0x558FE0A1D370	0xAB1FC143A6	0x7	miss
&b[8]	0x558FE0A1DC70	0xAB1FC143B8	0x7	miss
&a[0][9]	0x558FE0A1D378	0xAB1FC143A6	0x7	hit
&b[9]	0x558FE0A1DC78	0xAB1FC143B8	0x7	hit



NVIDIA Tegra Orin

- D-L1 Cache configuration of NVIDIA Tegra Orin
 - Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
}
```

What's the data cache miss rate for this code?

- A. 12.5%
- B. 56.25%
- C. 66.67%
- D. 68.75%
- E. 100%



NVIDIA Tegra Orin

100% miss rate!

- Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
```

C = ABS

32KB = 4 * 64 * S

S = 256

offset = lg(64) = 6 bits

index = lg(256) = 8 bits

tag = the rest bits

	Address (Hex)	Address in binary			Tag	Index	Hit? Miss?	Replace?
a[0]	0x10000	0b000100	00000000	000000	0x4	0x0	Miss	
b[0]	0x20000	0b001000	00000000	000000	0x8	0x0	Miss	
c[0]	0x30000	0b001100	00000000	000000	0xC	0x0	Miss	
d[0]	0x40000	0b010000	00000000	000000	0x10	0x0	Miss	
e[0]	0x50000	0b010100	00000000	000000	0x14	0x0	Miss	a[0-7]
a[1]	0x10008	0b000100	00000000	001000	0x4	0x0	Miss	b[0-7]
b[1]	0x20008	0b001000	00000000	001000	0x8	0x0	Miss	c[0-7]
c[1]	0x30008	0b001100	00000000	001000	0xC	0x0	Miss	d[0-7]
d[1]	0x40008	0b010000	00000000	001000	0x10	0x0	Miss	e[0-7]
e[1]	0x50008	0b010100	00000000	001000	0x14	0x0	Miss	a[0-7]
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮

Estimating code performance on caches (cont.)

NVIDIA Tegra Orin 100% miss rate!

- Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
```

C = ABS
 64KB = 4 * 64 * S
 S = 256
 offset = lg(64) = 6 bits
 index = lg(256) = 8 bits
 tag = the rest bits

	Address (Hex)	Address in binary			Tag	Index	Hit? Miss?	Replace?	
a[0]	0x10000	0	b000100	00000000	000000	0x4	0x0	Miss	
b[0]	0x20000	0	b001000	00000000	000000	0x8	0x0	Miss	
c[0]	0x30000	0	b001100	00000000	000000	0xC	0x0	Miss	
d[0]	0x40000	0	b010000	00000000	000000	0x10	0x0	Miss	
e[0]	0x50000	0	b010100	00000000	000000	0x14	0x0	Miss	a[0-7]
a[1]	0x10008	0	b000100	00000000	001000	0x4	0x0	Miss	b[0-7]
b[1]	0x20008	0	b001000	00000000	001000	0x8	0x0	Miss	c[0-7]
c[1]	0x30008	0	b001100	00000000	001000	0xC	0x0	Miss	d[0-7]
d[1]	0x40008	0	b010000	00000000	001000	0x10	0x0	Miss	e[0-7]
e[1]	0x50008	0	b010100	00000000	001000	0x14	0x0	Miss	a[0-7]
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮

NVIDIA Tegra Orin

- D-L1 Cache configuration of NVIDIA Tegra Orin
 - Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
}
```

What's the data cache miss rate for this code?

- A. 12.5%
- B. 56.25%
- C. 66.67%
- D. 68.75%
- E. 100%



intel Core i7 14700K

- D-L1 Cache configuration of intel Core i7 14700K's P-Core
- Size 48KB, 12-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
}
```

What's the data cache miss rate for this code?

- A. 12.5%
- B. 56.25%
- C. 66.67%
- D. 68.75%
- E. 100%

A screenshot of a Pollev poll interface. It shows a vertical list of five input boxes, each with a letter label (A, B, C, D, E) on the left. The boxes are currently empty, indicating that no answers have been submitted yet. The interface is clean and modern, with a light blue header and a white background for the poll area.

intel Core i7 14700K

- Size 48KB, 12-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
```

C = ABS
48KB = 12 * 64 * S
S = 64
offset = lg(64) = 6 bits
index = lg(64) = 6 bits
tag = the rest bits

	Address (Hex)	Address in binary	Tag	Index	Hit? Miss?	Replace?
a[0]	0x10000	0b0001000000000000000000	0x10	0x0	Miss	
b[0]	0x20000	0b0010000000000000000000	0x20	0x0	Miss	
c[0]	0x30000	0b0011000000000000000000	0x30	0x0	Miss	
d[0]	0x40000	0b0100000000000000000000	0x40	0x0	Miss	
e[0]	0x50000	0b0101000000000000000000	0x50	0x0	Miss	
a[1]	0x10008	0b0001000000000000001000	0x10	0x0	Hit	
b[1]	0x20008	0b0010000000000000001000	0x20	0x0	Hit	
c[1]	0x30008	0b0011000000000000001000	0x30	0x0	Hit	
d[1]	0x40008	0b0100000000000000001000	0x40	0x0	Hit	
e[1]	0x50008	0b0101000000000000001000	0x50	0x0	Hit	
⋮	⋮	⋮	⋮	⋮	⋮	⋮

intel Core i7 14700K (cont.)

- Size 48KB, 12-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
```

C = ABS
48KB = 12 * 64 * S
S = 64
offset = lg(64) = 6 bits
index = lg(64) = 6 bits
tag = the rest bits

	Address (Hex)	Address in binary	Tag	Index	Hit? Miss?	Replace?
a[7]	0x10038	0b00010000000000111000	0x10	0x0	Hit	
b[7]	0x20038	0b00100000000000111000	0x20	0x0	Hit	
c[7]	0x30038	0b00110000000000111000	0x30	0x0	Hit	
d[7]	0x40038	0b01000000000000111000	0x40	0x0	Hit	
e[7]	0x50038	0b01010000000000111000	0x50	0x0	Hit	
a[8]	0x10040	0b00010000000001000000	0x10	0x1	Miss	
b[8]	0x20040	0b00100000000001000000	0x20	0x1	Miss	
c[8]	0x30040	0b00110000000001000000	0x30	0x1	Miss	
d[8]	0x40040	0b01000000000001000000	0x40	0x1	Miss	
e[8]	0x50040	0b01010000000001000000	0x50	0x1	Miss	
a[9]	0x10048	0b00010000000001001000	0x10	0x1	Hit	
b[9]	0x20048	0b00100000000001001000	0x20	0x1	Hit	
c[9]	0x30048	0b00110000000001001000	0x30	0x1	Hit	
d[9]	0x40048	0b01000000000001001000	0x40	0x1	Hit	

$$\frac{5 \times \frac{512}{8}}{5 \times 512} = \frac{1}{8} = 12.5 \%$$

Miss when the array index is a multiply of 8!

Announcement

- Reading quiz due next Tuesday before the lecture
- **Assignment 2 released**
 - **Strongly encourage to practice Q1-Q9 before taking your midterm**
- **Assignment 1 due tonight**
 - We cannot help you at the last minute — please start early
 - Watch before you start https://youtu.be/m7OoY8y_Isk
 - Submit to the right item on Gradescope — Gradescope can give you feedback and you're allowed to resubmit
- Please make reservations for your 1st midterm starting from 8am this Thursday
 - <https://testcenter.cs.ucr.edu/Make-Change-Delete-Reservations>
 - First come first serve, not guaranteed to accommodate your preferred time if you book late
 - Slots span from Monday-Thursday next week
 - Arrive at WCH 133 early on your reserved slot
- Check our website for slides/quiz links, Gradescope to submit assignments, piazza for discussions
- Check your grades at https://www.escalab.org/my_grades
- Youtube channel for lecture recordings:
<https://www.youtube.com/c/ProfUsagi/playlists>

Sample midterm

Highlights about the midterm

- Multiple choices: 15
- Free answer/calculation based: 3
- WCH 133 at your chosen slot
- 80 minutes
- Closed book, closed notes

Demo — programmer & performance

A

```
for(i = 0; i < ARRAY_SIZE; i++)  
{  
    for(j = 0; j < ARRAY_SIZE; j++)  
    {  
        c[i][j] = a[i][j]+b[i][j];  
    }  
}
```

B

```
for(j = 0; j < ARRAY_SIZE; j++)  
{  
    for(i = 0; i < ARRAY_SIZE; i++)  
    {  
        c[i][j] = a[i][j]+b[i][j];  
    }  
}
```

How many of the following make(s) the performance different between version A & version B?

- ① IC
 - ② CPI
 - ③ CT
- A. 0
B. 1
C. 2
D. 3

How compilers affect performance

- If we turn on "-O3" in gcc when compiling both code snippets **A** and **B**, how many of the following can we expect?

- ① Compiler optimizations can reduce IC for both
- ② Compiler optimizations can make the CPI lower for both
- ③ Compiler optimizations can make the ET lower for both
- ④ Compiler optimizations can transform code B into code A

A. 0

B. 1

C. 2

D. 3

E. 4

A

```
for(i = 0; i < ARRAY_SIZE; i++)
{
    for(j = 0; j < ARRAY_SIZE; j++)
    {
        c[i][j] = a[i][j]+b[i][j];
    }
}
```

B

```
for(j = 0; j < ARRAY_SIZE; j++)
{
    for(i = 0; i < ARRAY_SIZE; i++)
    {
        c[i][j] = a[i][j]+b[i][j];
    }
}
```

Speedup of Y over X

- Consider the same program on the following two machines, X and Y. By how much Y is faster than X?

	Clock Rate	Dynamic Instruction	Percentage of Type-A	CPI of Type-A	Percentage of Type-B	CPI of Type-B	Percentage of Type-C	CPI of Type-C
Machine X	4 GHz	5000000000	20%	5	20%	2	60%	1
Machine Y	6 GHz	5000000000	20%	7	20%	2	60%	1

- A. 0.2
- B. 0.25
- C. 0.8
- D. 1.25
- E. No changes

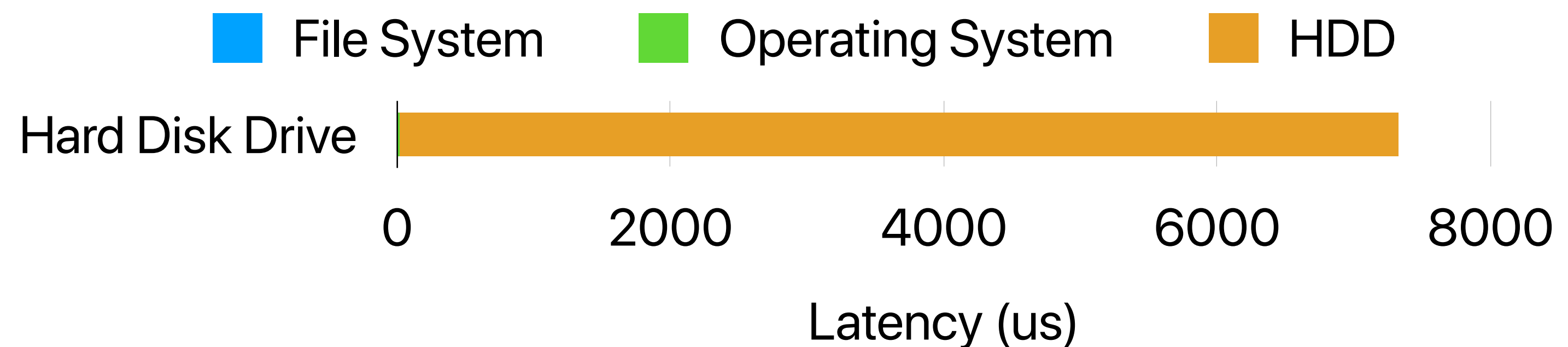
Clock rate and its affect on performance?

- What is true about clock rate and its affect on performance?
 - A. Modern processors allow users to freely adjust clock rates to achieve optimal performance for any application.
 - B. Increasing clock rate reduces instruction count.
 - C. The relationship between clock rate and performance is complex and can be influenced by other factors like CPI.
 - D. Increasing clock rate always leads to proportionally better performance.

Practicing Amdahl's Law

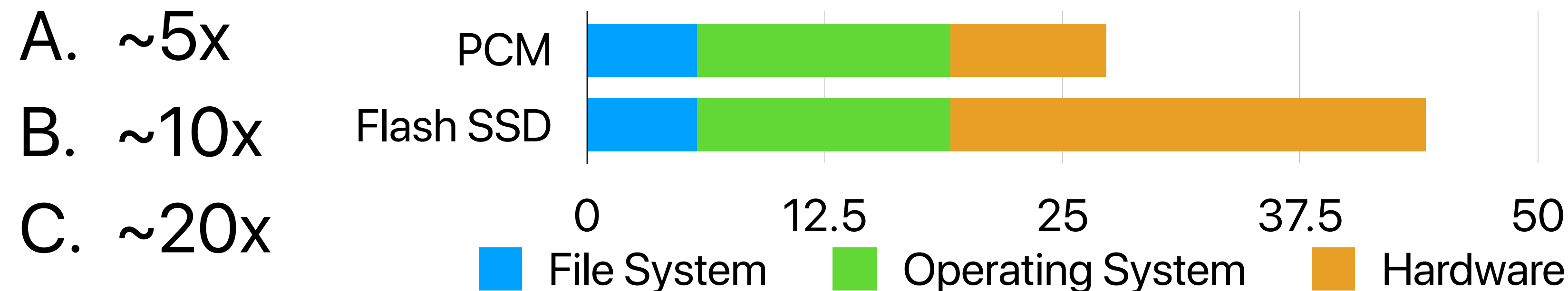
- Final Fantasy XV spends lots of time loading a map — within which period that 95% of the time on the accessing the H.D.D., the rest in the operating system, file system and the I/O protocol. If we replace the H.D.D. with a flash drive, which provides 100x faster access time. By how much can we speed up the map loading process?

- A. $\sim 7x$
- B. $\sim 10x$
- C. $\sim 17x$
- D. $\sim 29x$
- E. $\sim 100x$



Speedup further!

- With the latest flash memory technologies, the system spends 16% of time on accessing the flash, and the software overhead is now 84%. If your company ask you and your team to invent a new memory technology that replaces flash to achieve 2x speedup on loading maps, how much faster the new technology needs to be?



2x is not possible

Amdahl's Law on Multicore Architectures

- Regarding Amdahl's Law on multicore architectures, how many of the following statements is/are correct?
 - ① If we have unlimited parallelism, the performance of executing each parallel partition does not matter as long as the performance slowdown in each piece is bounded
 - ② With unlimited amount of parallel hardware units, single-core performance does not matter anymore
 - ③ With unlimited amount of parallel hardware units, the maximum speedup will be bounded by the fraction of parallel parts
 - ④ With unlimited amount of parallel hardware units, the effect of scheduling and data exchange overhead is minor
- A. 0
B. 1
C. 2
D. 3
E. 4

How reflective is FLOPS?

- If we're given the FLOPS of the underlying GPU of the machine, how many situations below can the FLOPS be representative to the real performance?
 - ① The FLOPS remains the same if we change the dataset for the same program
 - ② The FLOPS remains the same if we change the data type to double
 - ③ The FLOPS remains the same if we run a different the algorithm
 - ④ The FLOPS ratio of two computers running the same program will be the speedup of running the same program on these two computers
- A. 0
B. 1
C. 2
D. 3
E. 4

The impact of "slow" memory

- Assume that we have a processor running @ 4 GHz and a program with 20% of load/store instructions. If the instruction has no memory access and the processor already fetches the instruction, the CPI is just 1. Now, consider we have DDR5. The program is well-optimized so precharge is never necessary — the memory access latency is 13.75 ns. What's the average CPI (pick the closest one)?
 - A. 5
 - B. 10
 - C. 30
 - D. 50
 - E. 70

How can "memory hierarchy" help in performance?

- Assume that we have a processor running @ 4 GHz and a program with 20% of load/store instructions. If the instruction has no memory access, the CPI is just 1. Now, in addition to we DDR5, whose latency 13.75 ns, we also got an SRAM cache with latency of just at 0.5 ns and can capture **90%** of the desired data/instructions. what's the average CPI (pick the closest one)?
 - A. 5
 - B. 10
 - C. 30
 - D. 50
 - E. 70

How can a deeper memory hierarchy help in performance?

- Assume that we have a processor running @ 4 GHz and a program with 20% of load/store instructions. If the instruction has no memory access, the CPI is just 1. Now, in addition to the DDR5, whose latency is 13.75 ns, we also got 2 levels SRAM caches where
 - the 1st-level one at latency of 0.5ns and can capture 90% of the desired data/instructions.
 - the 2nd-level at latency of 5 ns and can capture 60% of the desired data/instructions

What's the average CPI (pick the closest one)?

- A. 5
- B. 10
- C. 30
- D. 50
- E. 70

What is the primary benefit of caching?

- What is the primary benefit of caching?
 - A. It can reduce IC
 - B. It can reduce CT
 - C. It can reduce CPI
 - D. It increases a program's inherent temporal locality.
 - E. It increases a program's inherent spatial locality.

Data locality

- Which description about locality of arrays `matrix` and `vector` in the following code is the **most accurate**?

```
for(uint64_t i = 0; i < m; i++) {  
    result = 0;  
    for(uint64_t j = 0; j < n; j++) {  
        result += matrix[i][j]*vector[j];  
    }  
    output[i] = result;  
}
```

- A. Access of `matrix` has temporal locality, `vector` has spatial locality
- B. Both `matrix` and `vector` have temporal locality, and `vector` also has spatial locality
- C. Access of `matrix` has spatial locality, `vector` has temporal locality
- D. Both `matrix` and `vector` have spatial locality and temporal locality
- E. Both `matrix` and `vector` have spatial locality, and `vector` also has temporal locality

NVIDIA Tegra Orin

- L1 data (D-L1) cache configuration of NVIDIA Tegra Orin
 - Size 64KB, 4-way set associativity, 64B block
 - Assume 64-bit memory address

Which of the following is correct?

- A. Tag is 49 bits
- B. Index is 8 bits
- C. Offset is 7 bits
- D. The cache has 1024 sets
- E. None of the above

intel Core i7 14700K

- L1 data (D-L1) cache configuration of Core i7 14700K's P-Core
 - Size 48KB, 12-way set associativity, 64B block
 - Assume 64-bit memory address
 - Which of the following is **NOT** correct?
 - A. Tag is 52 bits
 - B. Index is 6 bits
 - C. Offset is 6 bits
 - D. The cache has 128 sets
 - E. All of the above are correct

Performance equation

- Consider the following c code snippet and x86 instructions implement the code snippet
- If (1) `data_size` is set to 1,000,000,000,
(2) a memory instruction takes 5 cycles,
(3) a branch/jump instruction takes 2 cycles,
(4) other instructions takes 1 cycle on average,
and (5) the processor runs at 4 GHz,
how much time is it take to finish executing the code snippet?

C	x86
<pre>int init_data(int64_t *data, int data_size) { register unsigned int i = 0; for(i = 0; i < data_size; i++) { s+=data[i]; } return s; }</pre>	<pre>.LFB16: endbr64 testl %esi, %esi jle .L2 leal -1(%rsi), %ecx xorq %rax, %rax .L3: movslq (%rdi), %rdx addq \$4, %rdi addq %rdx, %rax cmpq %rcx, %rdi jne .L3 .L2: xorlq %rax, %rax ret</pre>

Practicing Amdahl's Law (2)

- After applying an SSD, Final Fantasy XV now spends 16% in accessing SSD, the rest in the operating system, file system and the I/O protocol. Please estimate the speedup of the followings.
 - A. Replacing the CPU to speed up the rest by 2x
 - B. Replacing the CPU to speed up the rest by 1.2x and replacing the SSD to speed up the SSD part by 100x
 - C. Replacing the CPU to speed up the rest by 1.5x and replacing the SSD to speed up the SSD part by 20x
 - D. Replacing the SSD to speed up the SSD part by 200x

intel Core i7 14700K

- D-L1 Cache configuration of intel Core i7 14700K's P-Core
 - Size 48KB, 12-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
}
```

What's the data cache miss rate for this code?

Computer Science & Engineering

203

つづく

